

中山大學

SUN YAT-SEN UNIVERSITY

数据库原理实验报告

二次元周边商品店数据库重构与应用实现

姓 名　郑立峰　李维政

学 号　24325341　24325143

学 院　计算机学院

专 业　计算机科学与技术

2026 年 10 月 24 日

目录

1 实验目标与阶段安排 **1**
1.1 从关系数据库到可操作应用 . . . 1
1.2 实现范围与交付内容 . . . 1
1.3 实验环境 . . . 1

2 第 5 周 ER 模型与业务规则检查 **1**
2.1 实体划分与联系表达 . . . 1
2.2 商品与库存的基数和参与约束 . . . 2
2.3 销售订单与人员联系 . . . 3
2.4 预售与采购规则及原结构问题 . . . 4

3 第 6 周函数依赖与规范化取舍 **5**
3.1 候选码与主要函数依赖 . . . 5
3.2 规范化示例与分解性质 . . . 6
3.3 保留字段的设计理由 . . . 6
3.4 快照，到货数量与约束的剩余边界 . . . 7

4 迁移方案与回归结果 **8**
4.1 三项结构调整及执行顺序 . . . 8
4.2 冲突处理与实际调试 . . . 8
4.3 迁移前后语义保持 . . . 9

5 第 7–9 周应用设计与实现 **10**
5.1 四层结构与模块职责 . . . 10
5.2 一条查询与一条修改的执行路径 . . . 11
5.3 身份与三层权限控制 . . . 12
5.4 草稿与结账的事务边界 . . . 13
5.5 统一异常与可追踪日志 . . . 14

6 本机业务流程与结果分析 **15**
6.1 商品维护与库存入库 . . . 15
6.2 草稿编辑，结账与数据库核对 . . . 16
6.3 两位顾客的本人订单查询 . . . 17
6.4 库存不足与整单回滚 . . . 18

7 集成测试，安全验证与复现 **19**
7.1 本机测试范围与实际结果 . . . 19
7.2 安全与失败场景的验证方法 . . . 19

8 分工，AI 辅助与实验总结 **20**
8.1 小组分工与辅助过程 . . . 20
8.2 实验认识与当前不足 . . . 20

1 实验目标与阶段安排

1.1 从关系数据库到可操作应用

第一阶段已经围绕单门店二次元周边商品店建立商品，库存，会员，员工，销售，预售和采购等关系。本阶段以该 v0.1 数据库为基础，先从现有表结构反推 ER 模型，再检查函数依赖和业务约束，形成第六周目标结构，最后通过 Web 程序完成登录，查询，维护和现货结账。

1.2 实现范围与交付内容

应用选择现货经营作为本次完整演示流程。店长维护商品，库存管理员调整库存，店员登记目录标价并编辑草稿订单，结账沿用明细单价，扣减库存并完成订单，顾客查询本人订单。原有预售，采购关系及样例数据继续保留，完整预售履约，采购验收和会员资料维护界面尚未实现。

第六周迁移保留原有 15 张业务表和 96 个字段。第 7–9 周为登录和访问控制新增技术表 `WebUser`，两个编号序列，五类应用数据库角色和顾客订单行级安全策略。

1.3 实验环境

实验使用 Python 3.12，Flask 和 SQL Server，通过 pymssql 连接数据库，Conda 管理运行环境。应用库为 `AnimeGoodsStoreDB_Week6App`，由基础建表与样例脚本，第六周迁移及应用安全脚本依次初始化。网页处理业务操作，SSMS 用于核对表结构与写入结果。

2 第 5 周 ER 模型与业务规则检查

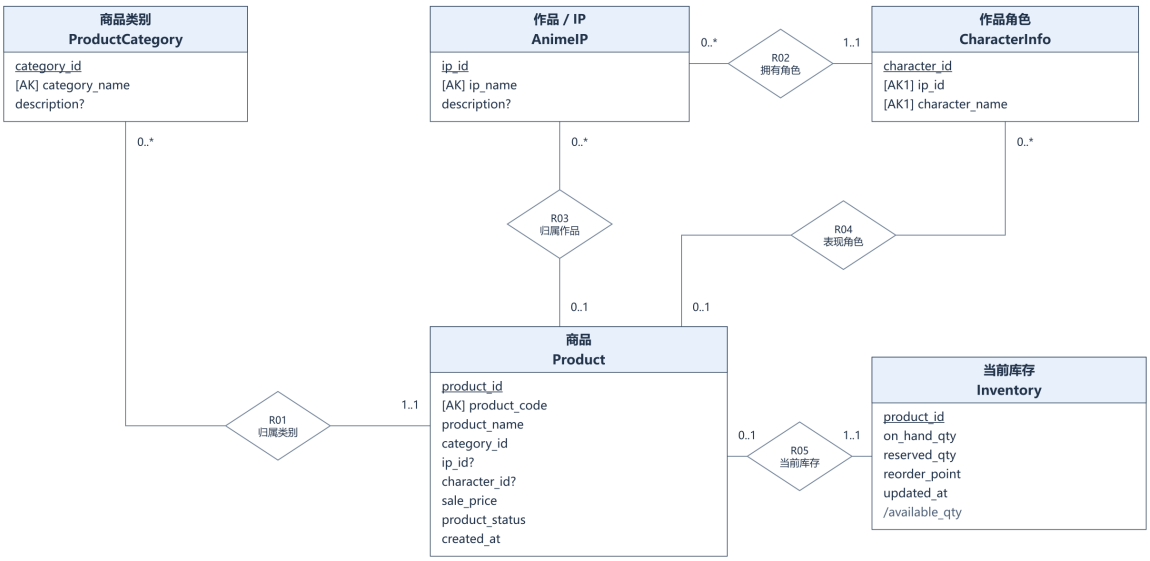
2.1 实体划分与联系表达

ER 模型从既有 15 张业务表反推实体和联系，按基础资料，当前库存与业务记录组织。商品类别，IP，角色和人员等资料独立保存，订单与采购通过明细关联商品。识别候选码时以业务约定和唯一约束为依据，不能因为样例中没有重复就将某列当作码。

2.2 商品与库存的基数和参与约束

每件商品必须归属一个类别，可以不指定 IP，也可以只指定 IP 而不指定具体角色。指定具体角色时必须指定其所属 IP，二者通过组合约束保持一致。这样既能表示单角色徽章，也能表示只关联作品的群像商品。

每条库存记录必须对应一件商品，库存以 `product_id` 同时作为主码和外码。同一商品至多有一条当前库存记录。原有结构允许商品先建档而尚无库存行，应用新增商品时进一步在同一事务中补建零库存记录，使后续库存查询与调整有明确的起点。



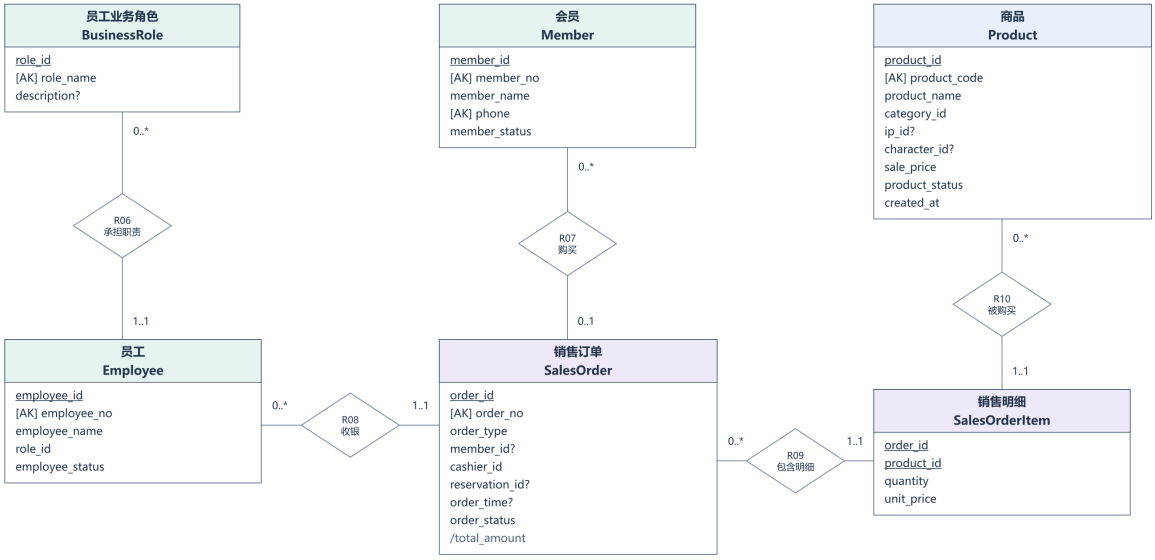
商品与库存模块 ER 图

图中实体旁的最小和最大次数表示该实体参与联系的次数，下划线表示主码，AK 表示候选码辅助标记，问号表示可空字段，斜线表示派生属性。可售数量由实际库存减预留库存得到，不重复存储一份可能失去同步的数值。

2.3 销售订单与人员联系

一笔销售订单由一位员工收银，可以关联一位会员，也可以是散客订单。一条明细必须同时属于一笔订单和一件商品，以 `order_id,product_id` 作为联合主码。同一商品在同一订单内合并为一行，数量和成交单价依赖整组主码。

订单与明细的基数需要结合状态理解。草稿允许暂时没有明细，完成时必须至少有一行。仅靠明细外码只能保证明细指向存在的订单，不能保证每笔完成订单非空，也不能自动禁止完成后的修改。因此，这部分规则必须在应用的状态检查和结账事务中继续实现。



销售，明细与人员模块 ER 图

业务角色表描述员工职责，网页登录角色用于决定允许执行的功能，二者通过后端身份校验对应。外码能够保证收银员工存在，但员工是否启用，是否承担正确职责，仍需由业务层检查。

2.4 预售与采购规则及原结构问题

预售活动对应一件商品，会员可以形成多次预订历史。取消后重新预订仍应保留记录，因此 `presale_id,member_id` 不能直接当作预订表候选码。一条预订最多对应一笔提货销售订单，提货需要与原会员和商品一致，不能仅凭一条外码认定整条履约链已经正确。

本组选择一场预售最多生成一张采购单，普通补货不关联预售活动。采购单明细描述采购商品和数量，供应商负责供货。完整联系与属性见提交目录中的 ER 图和数据字典，下面列出影响结构调整与应用实现的问题。

编号	原结构问题	处理位置与实际边界
P01	同一预售可能关联多张采购单。	第六周为非空活动编号增加筛选唯一索引，限制同活动最多一张采购单。
P02	状态包含部分到货，与整单到货约定不一致。	删除 `PARTIAL_RECEIVED` 状态，保留明细到货数量，数量与整单状态的联动仍未强制。
P03	完成订单可能无明细或继续被修改。	应用允许草稿编辑，结账拒绝空单，完成后交易内容只读。
P04	商品建档后可能没有库存行。	应用新增商品时同事务创建零库存行，库存调整也处理缺失记录。
P05	普通补货可能错误关联预售活动。	第六周以双向 CHECK 同时约束采购类型和活动编号。
P06	预售会员，商品，预订状态存在跨表规则。	明确列为后续完整预售履约工作，当前现货界面不宣称已完成。
P07	销售，库存与操作身份可能脱节。	当前现货结账通过登录身份，角色检查及统一事务实现，采购和预售链尚待扩展。

表 1: 由 ER 反查得到的问题与落实位置

检查的作用并非增加实体数量，而是区分能够通过键，外码和 CHECK 表达的规则，与需要多条语句共同完成的流程规则。后者不能只写在图旁注释中，还应在应用中找到对应实现和失败测试。

3 第 6 周函数依赖与规范化取舍

3.1 候选码与主要函数依赖

函数依赖由业务约定决定，而不是由某一批数据的取值偶然性决定。分析时先确定完整候选码，再检查非主属性是否依赖整组码，以及是否通过其他非主属性形成传递依赖。

关系	主要候选码	依赖与分析结论
`ProductCategory`	类别 ID，类别名称	任一候选码确定描述等属性，类别名称单独存储，避免在商品行内反复更新。
`CharacterInfo`	角色 ID，IP 与角色名称组合	角色 ID 确定 IP 及角色名称，辅助唯一组合 `character_id,ip_id` 是超码，但不是最小候选码。
`Product`	商品 ID，商品编号	商品码确定商品资料，在指定角色的子集内还存在角色 ID 决定 IP 的依赖，保留受控冗余。
`Inventory`	商品 ID	商品 ID 确定当前数量，预留数量，警戒值和更新时间，可售数量由查询计算。
`SalesOrder`	订单 ID，订单编号	订单码确定会员，收银员及状态，非空预订编号的条件唯一性不作为全表普通候选码。
`SalesOrderItem`	订单 ID 与商品 ID 组合	整组码确定数量和成交单价，不将商品当前售价误作历史成交价的决定因素。
`PresaleReservation`	预订 ID，预订编号	保留取消与重订历史，活动与会员组合不保证全表唯一。
`PurchaseOrderItem`	采购 ID 与商品 ID 组合	整组码确定采购量，成本和到货量，数量约束与状态联动需要分别判断。

表 2: 代表性关系的候选码与依赖分析

会员手机号在当前设计中受唯一约束，员工手机号及供应商名称，电话不据此认定为候选码。类别，IP，角色和人员资料与交易明细分离后，避免将商品名，会员姓名或类别名称复制到每一行明细中。订单明细中的数量和成交单价依赖联合主码的整体，没有将只依赖商品编号的商品基础资料混入明细。

3.2 规范化示例与分解性质

为说明更新异常及分解方法，采用一个用于理论分析的合并关系。

$$R(\textit{product_id}, \textit{product_name}, \textit{category_id}, \textit{category_name})$$

其主要函数依赖为商品 ID 决定商品名与类别 ID，类别 ID 决定类别名。若类别名随商品重复保存，修改一个类别名称需要修改多行，尚无商品的类别难以单独插入，删除某类别最后一件商品还可能同时丢失类别资料。

将其分解为商品关系和类别关系。

$$R_1 = (\textit{product_id}, \textit{product_name}, \textit{category_id})$$
$$R_2 = (\textit{category_id}, \textit{category_name})$$

两个关系的交集为类别 ID，它能够决定 R_2 的全部属性，因此该二元分解满足无损连接条件。商品 ID 决定商品名和类别 ID 的依赖保留在 R_1，类别 ID 决定类别名的依赖保留在 R_2，原依赖可以由分解后的局部依赖直接检查，因此依赖也得到保持。

这一示例用于解释现有类别表与商品表分离的理由，不是声称第六周实际又新建了两张表。原业务结构已经完成大部分实体拆分，本轮真正执行的迁移集中在采购约束，而不是为了体现规范化而重复拆表。

3.3 保留字段的设计理由

商品同时保留 IP 和具体角色。若只保留角色字段，将无法直接表示只有作品归属的群像商品。对于角色非空的记录，角色 ID 决定 IP，而角色 ID 既不是商品表超码，IP 也不是主属性，该条件子集存在不满足严格第三范式的依赖。当前选择保留两个字段，以组合外码和空值规则约束一致性。这是业务表达能力与冗余维护成本之间的取舍，不能将全部 15 张表一概描述为严格第三范式。

保留历史值，派生值尽量查询计算。明细成交单价保存结账时的交易事实，不能随着商品售价变化而改变。订单总额则由明细数量乘成交单价求和，不再作为独立业务字段保存。可售库存同样按实际库存减预留库存计算，避免维护多个代表同一当前状态的数值。

3.4 快照，到货数量与约束的剩余边界

预售活动中的决策需求量是某一时点的快照，不等同于随取消和重订变化的当前有效预订数量。计划采购量也不必固定等于需求快照加额外备货量，因为可能使用已有库存满足部分需求。因此，不能仅凭字段名称就将它们视为可以随时重算的重复列。

订金实收与退款额按交易历史保留。如果额外假设实收额始终严格等于数量乘固定订金单价，则会出现可计算冗余，需进一步约束或调整。当前分析将这一假设单独说明，没有据此宣称已经完全消除了所有传递依赖。

采购明细继续保留 `received_qty`。在整单收货且始终严格执行状态联动的假设下，该值可能由状态和采购量推导，但当前数据库仅限制其位于零到采购量之间，没有强制所有行与整单状态同步。直接删除字段会改变原操作与查询含义，因此本轮先保留，并将跨表一致性列为后续流程工作。

选择	采用理由	需要承担的维护责任
保留商品 IP 与角色	支持群像商品及角色商品的不同表达。	校验角色归属 IP，避免不匹配。
保留明细成交价	保存交易历史，允许之后调整商品售价。	登记时保存目录标价，结账沿用，完成后禁止修改。
查询计算总额与可售量	数值由现有记录确定，避免重复维护。	查询与页面使用一致计算口径。
保留需求决策快照	保存采购决策时依据，便于追溯。	区分快照与当前汇总值。
暂保留采购到货量	保持原数据与操作含义。	后续实现数量与收货状态的事务联动。

表 3: 结构保留与受控冗余的取舍

本轮结论是保留 15 张业务表，不执行新的表拆分或合并，将采购规则从文档落实为约束。范式分析负责解释数据为何这样组织，实际迁移则必须以明确存在的问题和可验证的收益为依据。

4 迁移方案与回归结果

4.1 三项结构调整及执行顺序

迁移在独立基线副本中进行，不直接修改原实验库。脚本先核对当前库名，表，视图，角色及预期旧约束，再检查冲突记录，最后在同一事务中创建索引并替换 CHECK。发现冲突时列出记录并停止，不擅自改写既有业务事实。

```
CREATE UNIQUE INDEX UX_PurchaseOrder_PresaleID
ON dbo.PurchaseOrder(presale_id)
WHERE presale_id IS NOT NULL;

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_TypePresale CHECK
((purchase_type='PRESALE' AND presale_id IS NOT NULL)
 OR (purchase_type='RESTOCK' AND presale_id IS NULL));

ALTER TABLE dbo.PurchaseOrder WITH CHECK
ADD CONSTRAINT CK_PurchaseOrder_Status CHECK
(purchase_status IN ('CREATED','ORDERED','RECEIVED','CANCELLED'));
```

以上为迁移的目标约束片段，实际脚本还包含旧约束移除，冲突检查，事务与异常回滚。筛选唯一索引只约束非空活动编号，允许多张普通补货单的活动编号为空。它同时约束已取消采购单，因此当前规则仍是同活动最多一张采购记录，并未开放取消后另建采购单。

采购类型检查从原来的单向要求扩展为双向对应，预售采购必须关联活动，普通补货必须不关联活动。状态域移除部分到货，只限制允许保存的状态文字，不自动保证明细到货量与状态相符。

脚本启用 XACT_ABORT，设置筛选索引所需的会话选项，并使用 WITHCHECK 验证既有行。旧结构识别失败或重复执行会停止，防止把一次性迁移当作可反复初始化脚本。

4.2 冲突处理与实际调试

冲突测试覆盖同活动多单，采购类型关联不符和非法状态，均停止迁移且不改写数据。对已迁移副本再次执行返回 51005。曾因筛选索引会话选项不一致出现 1934 错误，补齐 SET 并关闭 NUMERIC_ROUNDABORT 后，45 项回归通过，应用连接沿用相同选项。

4.3 迁移前后语义保持

回归分类	通过数	检查内容
DATA	15	全部业务表的行数及全列双向 EXCEPT，检查数据是否增减或改变。
QUERY	5	订单明细，金额，销量，库存等关键查询，检查业务结果是否保持。
META	7	旧结构与目标结构的键，索引，约束等元数据。
TARGET	1	新增筛选唯一索引和两项 CHECK 均启用且受信任。
WRITE	4	合法写入仍然可以执行。
REJECT	7	重复活动采购，类型不符，非法状态等反例被拒绝。
ROLE	6	既有员工角色的允许与拒绝行为。
合计	45	45 项通过，0 项失败。

表 4: 第六周独立副本回归结果

核对项目	迁移前	迁移后	解释
SO004 金额	124.00	124.00	两条明细汇总保持。
6 笔订单金额合计	530.00	530.00	关键查询语义保持。
6 件商品可售库存合计	42	42	数量为 8，5，2，12，6，9。

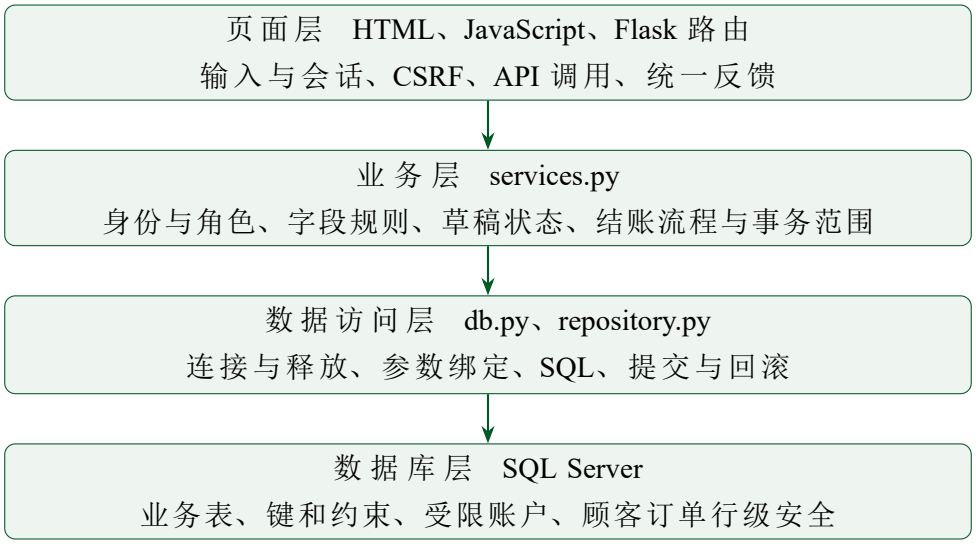
表 5: 固定样例的迁移前后对照

合法数据仍能保存，新增规则能拒绝对应反例，原记录与查询结果保持，这三者共同构成迁移验证。仅比较表行数不足以排除字段内容变化，仅看到新约束存在也不足以说明规则已经生效。完整 CSV 结果见提交目录的迁移与回归结果，regression.sql 位于代码目录。

5 第 7–9 周应用设计与实现

5.1 四层结构与模块职责

应用使用 Flask 接收页面和 API 请求，业务规则集中在服务层，所有运行时 SQL 收口到数据访问层。前端负责表单，操作入口和结果提示，不直接访问数据库，也不决定登录者的真实角色。



应用分层及请求流向

模块	负责内容
`app/templates/index.html`	页面结构，表格与对话框。
`app/static/app.js`	查询，表单提交，角色入口和错误展示。
`app/__init__.py`	路由，会话，CSRF，统一响应与异常映射。
`app/services.py`	权限，输入与业务状态校验，组织完整业务操作。
`app/db.py`	建立同事务连接，提交，回滚和关闭。
`app/repository.py`	参数化 SQL 与结果读取。
`sql/01_web_security.sql`	登录技术表，受限角色与顾客行级过滤。

表 6: 各层在源代码中的位置

5.2 一条查询与一条修改的执行路径

商品查询由页面读取搜索词并发送请求，Flask 路由读取会话身份，服务层确认允许查询并限制搜索词长度，Repository 执行带参数的查询，数据库返回商品与库存。服务层返回结构化数据，页面刷新表格和统计卡片。

```
# services.py
with self.connection(user) as repo:
    return repo.products("%" + term + "%", user["role"] in STAFF)
```

```
-- repository.py 中查询的参数绑定形式
WHERE (p.product_name LIKE %s OR p.product_code LIKE %s)
```

搜索词中的通配模式作为参数值传入，SQL 结构保持固定。上面的字符串连接构造的是搜索参数，不是把用户输入拼接为可执行 SQL。商品名，价格，数量，会员编号等写入值同样通过参数绑定传递。

商品新增由店长提交表单，服务层先验证编号，名称，类别，IP 与角色关系，价格和状态，再通过同一个事务连接写入商品及零库存记录。任一步骤失败，两项写入一起回滚。修改商品只允许管理员，删除还需检查库存和历史引用，不能为了满足删除按钮而破坏已成交数据。

功能	主要输入	处理与输出
商品查询与维护	搜索词，编号，名称，价格，类别等。	校验输入与角色，新增时创建零库存，返回商品编号并刷新列表。
库存调整	商品 ID，增减数量，警戒值。	限制整数范围与预留保护，返回当前数量和可售量。
草稿订单与明细	会员 ID，商品 ID，数量。	校验会员与订单状态，创建或更新明细，返回草稿金额。
确认结账	草稿订单 ID。	锁定并复核商品与库存，沿用登记单价，同事务完成扣减和状态变更。
顾客订单查询	会话中的顾客身份。	后端取会员绑定关系，只返回本人订单和明细。

表 7: 主要功能的输入，规则和结果

5.3 身份与三层权限控制

网页登录后，服务器将账户映射到员工或会员，重新检查其启用状态和员工职责。身份来自服务器会话和数据库绑定，不能由浏览器提交一个角色名称来取得权限。密码以哈希保存，不把明文密码写入业务表或报告。

角色	商品	库存	订单
店长 `manager`	查询，新增，修改，条件删除。	查询与调整。	创建草稿，编辑明细，结账，取消及草稿删除。
店员 `clerk`	查询。	在目录查看可售量。	创建草稿，编辑明细，结账，取消及草稿删除。
库存员 `inventory`	查询。	查询与调整。	不开放订单和会员查询。
顾客 `customer`	查询在售商品。	仅随目录查看可售量。	查询本人订单，不开放经营写入。

表 8: 四类业务角色的主要操作范围

店长的商品管理维护资料，售价和上架状态，库存管理单独展示及调整数量。店员只保留商品目录和销售订单，在目录查询可售量。库存员仅使用库存管理，顾客只看在售商品与本人订单。订单页按订单状态统计，顾客目录不展示经营指标。页面按职责提供入口，服务层逐项检查权限，数据库连接再选择对应的受限账户。配置中分为 `auth`，`manager`，`clerk`，`inventory` 和 `customer` 五种数据库身份，认证账户仅完成登录所需读取，网页运行不使用管理账户。

顾客存在两个演示用户 `customer1` 和 `customer2`，它们共享顾客功能范围，但绑定不同会员。顾客连接设置只读会话上下文 `member_id`，SQL Server 依据安全谓词过滤订单及明细。API 的会员过滤与数据库 RLS 共同限制横向访问，不能仅靠页面没有显示其他订单就认定安全。

会话请求同时校验 CSRF，商品名，价格和数量由后端再次验证。价格采用十进制处理并限制正值，两位小数和上限，数量拒绝零，负数，布尔值和非整数。前端校验用于改善操作反馈，后端校验决定输入是否可以进入业务流程。

5.4 草稿与结账的事务边界

订单先保存为 CREATED 草稿，允许调整会员和明细，草稿阶段不扣减库存。商品首次登记时保存目录标价，修改数量，刷新和结账都保留该单价。页面标价过期时拒绝新登记，提示刷新核对。结账扣减库存并设为 COMPLETED，目录之后调价不影响已有明细。完成和取消的记录保留为历史，不再开放交易内容编辑。草稿物理删除与取消是两个不同动作。

结账服务先锁定并检查订单状态和有效会员，拒绝空明细，再按商品 ID 顺序检查商品与库存。商品必须在售，数量必须有效，可售库存必须充足。所有明细共用同一个数据库连接和事务，不能在循环中每处理一件商品就单独提交。

```
UPDATE dbo.Inventory
SET on_hand_qty = on_hand_qty - %s,
    updated_at = SYSDATETIME()
WHERE product_id = %s
  AND on_hand_qty - reserved_qty >= %s;
```

扣减使用带可售量条件的原子 UPDATE，并检查实际影响行数。这样可以避免先读取库存，之后无条件扣减造成的竞争窗口。锁定与固定商品处理顺序进一步配合并发结账，测试验证两个竞争订单不会共同超卖。

```
# db.py 中事务管理的核心结构
try:
    # connection 已建立，cursor 已设置所需 SET 选项
    yield Repository(cursor)
    connection.commit()
except BaseException:
    connection.rollback()
    raise
finally:
    connection.close()
```

若后面的商品库存不足，即使前面的商品已经在事务内执行了扣减，也会整体回滚。库存扣减与订单状态变更必须一起成功或一起撤销，失败后登记单价保持不变。数据库约束错误，业务异常和执行失败均沿同一回滚路径返回上层，连接最后关闭。

5.5 统一异常与可追踪日志

后端统一处理错误，返回错误码，简明提示，数据和请求编号，前端不直接呈现 SQL Server 原始错误。业务方法通过统一异常对象表达失败，数据库异常在统一入口转换，避免各页面自行解释数据库报错而形成不同口径。

失败类别	HTTP 状态	处理方式
输入不符合要求	400	提示字段或数值要求，不继续写入。
未登录或身份失效	401	返回认证提示。
角色或 CSRF 检查失败	403	拒绝操作，不以隐藏按钮代替检查。
对象不存在	404	返回明确的资源不存在提示。
业务规则或约束冲突	409	返回库存不足，状态冲突等友好提示。
数据库连接失败	503	返回暂时不可用提示，详细原因留在日志。
未预期内部错误	500	返回统一错误，避免暴露堆栈和连接细节。

表 9: 统一错误分类与反馈

日志记录时间，请求编号，操作，角色，错误码和结果，必要时保留异常细节。页面上的请求编号用于定位同一次失败，不向顾客展示服务器路径，连接密码或数据库原始堆栈。日志和本地连接配置在仓库中被忽略，不作为普通业务文件提交。

本次库存不足操作返回 `INSUFFICIENT_STOCK` 和 409 状态，日志记录对应的店员结账请求，可通过页面提示中的请求编号定位。

6 本机业务流程与结果分析

6.1 商品维护与库存入库

店长新增商品 DEMO001 后，将名称改为初音未来徽章，售价改为 120 元，类别为徽章，IP 和角色均选择初音未来。另一件商品为 DEMO002，售价 20 元，名称实际保存为文本 None。新增和修改后可在目录查询，商品创建时自动生成零库存记录。



商品新增与修改表单，局部截图

切换到库存管理员，为两件商品分别增加 10 件和 5 件，警戒值设为 3 和 2，查询时可售量合计 15。库存员可以调整库存，但不能维护商品或查看订单。店员在商品目录查询售价与可售库存，手工库存调整由店长或库存员完成。



入库后两件商品库存为 10 和 5，局部截图

6.2 草稿编辑，结账与数据库核对

店员为会员 M001 创建 WEB0000010015，加入两件 DEMO001 和一件 DEMO002，金额为 260 元。将第一行数量改为 1 后，页面金额变为 140 元，说明修改明细后汇总随之更新。草稿阶段库存不变，最终将第一行恢复为 2 件再结账。

结账后订单显示已完成，两行金额分别为 240 元和 20 元，总额 260 元，明细变为只读。库存从 10 和 5 变为 8 和 4，与两行销售数量一致。



结账后保存成交价格与订单明细，局部截图

在 SSMS 中关联查询商品，库存，订单与明细，状态为 COMPLETED，会员 ID 为 1，数量和单价均与网页一致。

Results Messages

	product_code	product_name	on_hand_qty	reserved_qty	available_qty
1	DEMO001	初音未来徽章	8	0	8
2	DEMO002	None	4	0	4

	order_no	order_status	member_id	product_code	quantity	unit_price	line_amount
1	WEB0000010015	COMPLETED	1	DEMO001	2	120.00	240.00
2	WEB0000010015	COMPLETED	1	DEMO002	1	20.00	20.00

数据库中的库存与成交明细，局部截图

6.3 两位顾客的本人订单查询

登录 `customer1` 后，本人订单列表包含刚完成的 260 元订单，以及样例 SO004 和 SO003。订单详情只读，页面不开放商品维护，库存管理和经营订单写入。



顾客 1 可以查看会员 1 的订单，局部截图

退出后登录 `customer2`，列表只有会员 2 的 SO002，金额 90 元，没有会员 1 刚完成的订单。角色相同但绑定会员不同，查询结果也随之改变，后端与数据库的跨会员访问检查见集成测试。



顾客 2 只显示会员 2 的订单，局部截图

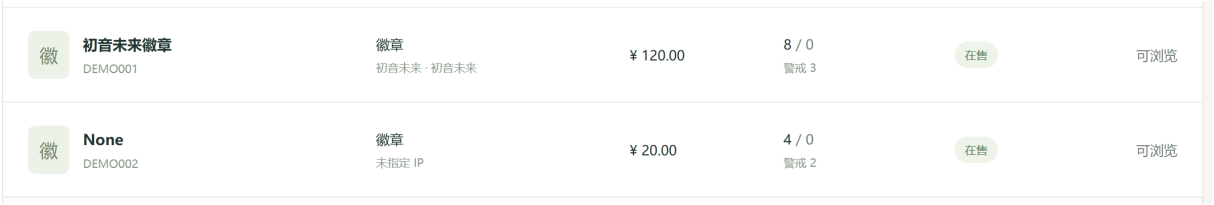
6.4 库存不足与整单回滚

店员另建草稿 WEB0000010016，第一行购买一件 DEMO001，第二行购买十件 DEMO002，金额 320 元。两件商品可售库存为 8 和 4，第二行超过库存，确认结账后提示库存不足，订单仍为草稿。



库存不足时拒绝结账并保留草稿，局部截图

重新查询，库存仍为 8 和 4。结账按商品 ID 顺序处理，第一件原本可以扣减，第二件失败后整单回滚，没有留下第一件减少而订单未完成的状态。日志中也能通过页面提示的请求编号找到该次失败。



失败后两件商品库存保持 8 和 4，局部截图

失败草稿可以修改后重试，也可以确认删除。本次删除后查询数据库，该订单编号已不存在。完成订单则保留交易历史，禁止继续修改。商品与明细的其他删除正例，引用冲突和权限反例由自动化测试补充验证。

7 集成测试，安全验证与复现

7.1 本机测试范围与实际结果

执行 pytest 后，48 项测试全部通过，0 失败，0 错误，0 跳过。其中 42 项为连接真实 SQL Server 的应用集成测试，另外 6 项为密码重设辅助脚本测试。

测试范围	代表性检查	结果
登录与请求保护	登录退出，未登录访问，密码哈希与后端身份，CSRF 与非法 JSON。	通过
商品与库存	商品 CRUD，自动零库存，重复约束回滚，非法价格，预留保护，有库存和有引用商品删除拒绝。	通过
订单与事务	草稿及明细 CRUD，空单拒绝，目录标价核对，草稿锁价，多商品结账，历史价保持，库存不足整单回滚。	通过
角色与顾客隔离	API 越权拒绝，顾客跨会员访问与 SQL RLS，受限 SQL 账户直接执行拒绝。	通过
并发与故障	并发结账不超卖，数据库连接失败与日志，页面安全响应头。	通过
密码重设辅助	生成哈希匹配且不含明文，非法输入拒绝，已有输出文件保留。	通过

表 10: 本机 48 项测试的范围与结果

浏览器验证中，商品按 25 元登记后目录调为 30 元，刷新和改数量仍保留 25 元，两件结账 50 元，库存由 10 减到 8。集成测试拒绝过期标价登记，临时数据清理后原演示记录保持一致。

7.2 安全与失败场景的验证方法

参数化查询测试使用包含引号和 SQL 片段的搜索词，检查输入按普通查询条件处理，结合 Repository 中的参数绑定确认 SQL 结构没有被改变。

权限测试覆盖顾客新增商品，店员调整库存，库存员查询订单和顾客跨会员访问的拒绝结果。受限 SQL 账户越权返回 229，数据库 RLS 进一步过滤本人订单。

异常测试检查错误码和状态，响应不含 SQL 与堆栈，日志可按请求编号定位。多商品结账失败后核对库存与状态回滚，登记单价不变。两笔订单并发争用同一库存时，核对成功笔数与最终数量，确认没有超卖。

8 分工，AI 辅助与实验总结

8.1 小组分工与辅助过程

郑立峰负责第 5–6 周的 ER 模型，业务规则，问题清单，依赖与范式分析，采购约束迁移及回归材料。李维政负责第 7–9 周的分层应用，数据库连接，角色界面，现货结账事务，异常处理，集成测试，本机演示与截图，并据此整合第二阶段报告。最终提交前由小组共同审阅各周材料与报告的一致性。

AI 辅助用于整理课程任务，检查设计依赖与约束表达，组织代码和测试，排查连接环境，梳理报告结构与图文说明。小组负责确定业务范围与保留字段的取舍，实际运行应用和 SQL，执行角色切换，数量修改，结账及失败场景，并保存截图。报告中的商品价格，订单数量，金额与库存采用实际截图和只读查询核对，不直接照抄预设演示值。具体过程与验证结论见提交目录中的 `ai_log.md`。

8.2 实验认识与当前不足

本阶段使数据设计从静态表结构进入可操作流程。ER 检查揭示了一些不能由外码自动表达的规则，函数依赖分析说明了基础资料分离和历史值保留的原因，迁移则把能够落实的采购规则写入数据库约束。结构调整并不一定意味着拆出更多表，保留字段同样需要明确业务含义和维护责任。

应用实现进一步表明，草稿和完成状态应有不同的操作范围。草稿允许反复编辑，登记时须保存与目录一致的单价，结账按该价格统一处理库存和状态。人工失败案例中，两件商品库存均保持不变，比单独展示一次成功扣减更直接地说明了事务边界的作用。

权限控制也需要贯穿界面，服务和数据库。不同角色看到的按钮只是操作体验，接口拒绝和受限账户才构成更完整的执行约束。对于顾客数据，还必须检查本人记录的范围，不能将是否登录等同于可以读取所有订单。

当前版本已经形成可运行的本机现货流程和可复现测试材料，完整预售履约，采购验收，到货数量与整单状态联动仍是后续工作。